

NerdVault

Relazione tecnica sull'architettura del sistema

Applicazione web marketplace — analisi di architettura, livelli e flussi di controllo

Stack: PHP 8 · MySQL/MariaDB · PDO · Smarty · Composer · Apache

Architettura MVC a livelli con Front Controller · deploy su hosting in ambiente di produzione

1. Introduzione e dominio applicativo

NerdVault è un'applicazione web di tipo **marketplace** per la compravendita di oggetti da collezione tra utenti. Il sistema gestisce annunci, ricerca, wishlist, carrello, pagamenti (simulati), recensioni, segnalazioni e un'area di amministrazione. È realizzata in PHP senza framework applicativi: l'intera infrastruttura (routing, persistenza, sessioni, sicurezza) è implementata esplicitamente, rendendo il progetto adatto a illustrare i principi architetturali sottostanti.

L'ambiente locale (XAMPP) è impiegato in fase di sviluppo, mentre il sistema è destinato alla pubblicazione in rete presso un **provider di hosting**: l'applicazione è quindi raggiungibile via Internet su un dominio pubblico in **HTTPS**. Le scelte progettuali descritte nel seguito (separazione della configurazione, restrizione del document root, cookie Secure) sono orientate proprio a un esercizio sicuro in ambiente di produzione (cap. 11).

Il dominio prevede tre attori, con responsabilità e privilegi disgiunti:

Attore	Descrizione	Operazioni consentite
Utente registrato	Soggetto privato	Pubblica annunci, ricerca, wishlist, carrello, acquisto, feedback.
Account business	Venditore professionale	Solo vendita: pubblicazione annunci e gestione ordini ricevuti.
Amministratore	Staff (due livelli di privilegio)	Moderazione di utenti, annunci e segnalazioni; audit delle azioni.

Tecnologie

- **PHP 8** per la logica server-side; **Composer** per autoload PSR-4 e dipendenze.
- **MySQL/MariaDB** come RDBMS, interrogato tramite **PDO** con prepared statement.
- **Smarty** come template engine per il presentation layer.
- **Apache** come web server, sia in locale sia presso il provider di hosting; il document root è ristretto alla cartella `public/`.

2. Architettura: pattern MVC a livelli

L'applicazione adotta il pattern architetturale **Model-View-Controller** combinato con un **Front Controller** unico e con una stratificazione in package a responsabilità singola. La separazione tra logica di presentazione, logica di coordinamento e logica di persistenza favorisce manutenibilità, testabilità e basso accoppiamento.

Livello	Package	Responsabilità
Core	<code>src/Core/</code>	Front Controller, gestione sessione, protezione CSRF, astrazione della richiesta.
Middleware	<code>src/middleware/</code>	Politiche di autorizzazione applicate prima dell'esecuzione del controller.
Controller	<code>src/controllers/</code>	Coordinamento della richiesta: validazione input, orchestrazione, selezione vista.
Model – Entity	<code>src/Entity/</code>	Oggetti di dominio (E*): stato e regole intrinseche dell'entità.
Model – Foundation	<code>src/Foundation/</code>	Data mapper (F*): SQL, mapping riga-entità, transazioni.
View	<code>src/View/</code> + <code>templates/</code>	Configurazione di Smarty e rendering dei template .tpl.
Services	<code>src/services/</code>	Servizi applicativi trasversali (invio email, eccezioni di dominio).

Principio progettuale fondamentale

I controller non contengono codice SQL. Ogni accesso ai dati è mediato dalla facciata statica `FPersistentManager`, che disaccoppia il livello di coordinamento dal livello di persistenza. Le query risiedono esclusivamente nelle classi Foundation.

3. Ciclo di vita di una richiesta

Ogni richiesta HTTP attraversa la medesima pipeline. L'unico punto di ingresso pubblico è `public/index.php`, che inizializza sessione e connessione al database e delega al Front Controller; il resto del codice in `src/` non è raggiungibile direttamente dal web.

Pipeline di elaborazione, identica per ogni rotta dell'applicazione:

```
HTTP request (HTTPS)
-> public/index.php          bootstrap: SessionManager + FDataBase::init(PDO)
-> FrontController          normalizzazione URL e risoluzione della rotta
-> [metodo HTTP] [CSRF] [middleware] verifiche preliminari
-> Controller::azione        validazione input e orchestrazione
-> FPersistentManager -> F* -> FDataBase -> DBMS
-> SmartyView::render        idratazione del template
-> HTTP response (HTML)
```

4. Front Controller e routing

La classe `App\Core\FrontController` implementa il dispatching. Normalizza il percorso della richiesta (rimozione di prefissi locali, di `index.php` e uniformazione), lo scompone in segmenti secondo lo schema `/controller/azione/parametri` e lo confronta con una **tabella delle rotte** dichiarativa. La risoluzione procede dal percorso più specifico, consentendo parametri posizionali (es. l'identificativo in `/annuncio/show/1`). L'inoltro delle richieste a `index.php` avviene tramite il file `public/.htaccess` di Apache.

Ciascuna voce della tabella associa al percorso il controller, l'azione, i metodi HTTP ammessi, i middleware e una funzione di estrazione dei parametri. Prima di invocare l'azione, il dispatcher applica in sequenza i seguenti controlli:

- 1 Risoluzione della rotta:** percorso non mappato → risposta 404.
- 2 Metodo HTTP:** le azioni con effetti collaterali ammettono solo POST; metodo non consentito → 405.
- 3 Token CSRF:** validato su ogni POST; assente o non valido → 400.
- 4 Middleware:** esecuzione delle politiche di autorizzazione associate alla rotta.
- 5 Invocazione:** chiamata del controller con i parametri estratti.

L'invocazione è mediata da *reflection* per verificare che il metodo bersaglio sia pubblico e non appartenga alla classe base, impedendo l'esposizione accidentale di metodi interni come azioni.

5. I controller

Ogni controller aggrega le azioni di un'area funzionale ed estende `BaseController`, che centralizza operazioni ricorrenti: rendering della vista, validazione degli identificativi, gestione degli errori HTTP e registrazione tracciata delle azioni di moderazione.

Controller	Responsabilità
HomeController	Home page con annunci in evidenza e raccomandazioni.
AnnuncioController	Elenco, ricerca, dettaglio, CRUD annunci e gestione immagini.
UtenteController	Registrazione, autenticazione, verifica email, recupero password, profilo, indirizzi.
WishlistController / CarrelloController	Gestione di preferiti e carrello dell'acquirente.
PagamentoController	Checkout singolo e da carrello, conferma del pagamento simulato, esito.
BusinessController	Creazione account business, dashboard e ordini ricevuti.
FeedbackController	Recensioni post-acquisto e reputazione del venditore.

Controller	Responsabilità
SegnalazioneController	Invio segnalazioni e relativa gestione amministrativa.
AdminController	Dashboard, gestione utenti e segnalazioni, moderazione.
LegalController	Pagine informative (privacy, termini, cookie).

Esempio — AnnuncioController::lista()

L'azione legge gli eventuali criteri di filtro (testo, categoria) dalla richiesta, ottiene dal Persistent Manager gli annunci attivi o i risultati della ricerca, integra — per i soli acquirenti — lo stato di wishlist e carrello, e delega il rendering al template `annunci/lista.tpl`. Il controller ignora sia la formulazione SQL sia il markup HTML: ne è responsabile, rispettivamente, il livello Foundation e il livello View.

6. Middleware e controllo degli accessi

I middleware sono funzioni di autorizzazione eseguite prima del controller. In caso di esito negativo interrompono la richiesta (tipicamente con un redirect), impedendo l'esecuzione dell'azione. Implementano la separazione netta dei privilegi tra i tre attori del dominio.

Middleware	Vincolo imposto
requireAuth	Utente autenticato; altrimenti redirect al login.
requireGuest	Utente non autenticato (login, registrazione, recupero password).
requireAdmin	Privilegi di amministratore.
requireAdminLivello2	Amministratore di livello 2 (azioni su altri amministratori).
requireBusiness	Account business attivo (dashboard, ordini).
denyAdmin	Esclude l'amministratore dalle funzioni di acquisto/vendita.
denyBusiness	Esclude l'account business da carrello e wishlist.

Disgiunzione dei ruoli

Le rotte di acquisto compongono più politiche — ad esempio `requireAuth + denyAdmin + denyBusiness` — per garantire che soltanto un utente acquirente possa accedervi. I middleware `deny*` formalizzano i vincoli di dominio “il business solo vende” e “l'amministratore solo modera”.

7. Livello di persistenza

Il Model è articolato in due gerarchie cooperanti, secondo una variante del pattern *Data Mapper*.

7.1 Entity (prefisso E)

Le classi `E*` modellano le entità di dominio (`EAnnuncio`, `EUtenteRegistrato`, `EPagamento`, ...): incapsulano stato in attributi privati con accessori e logica intrinseca (p.es. la verifica dello stato di un annuncio). Estendono `EBaseEntity`, che fornisce la conversione bidirezionale tra oggetto e array (`fromArray / toArray`).

7.2 Foundation (prefisso F)

Le classi `F*` mappano una tabella sulla rispettiva Entity ed estendono `FBaseTable`, che implementa CRUD generico, esecuzione parametrizzata e idratazione delle righe. Tre componenti infrastrutturali completano il livello:

Componente	Ruolo
FDataBase	Wrapper singleton della connessione PDO; espone le primitive transazionali e istanzia le classi Foundation condividendo la connessione.
FBaseTable	Classe astratta base dei mapper: query parametrizzate, white-list delle colonne scrivibili, validazione degli identificatori SQL, mapping verso Entity.
FPersistentManager	Facciata statica e unico punto di contatto dei controller con la persistenza; instrada le richieste alla Foundation appropriata.

Gestione transazionale

Le operazioni composite sono incapsulate in transazioni con rollback automatico. La conferma di un acquisto, ad esempio, esegue atomicamente: creazione del pagamento, marcatura dell'annuncio come venduto e rimozione dell'annuncio da carrelli e wishlist. La lettura dell'annuncio avviene con blocco `SELECT ... FOR UPDATE`, prevenendo la doppia vendita in condizioni di concorrenza.

7.3 Misure di robustezza dello strato dati

- Prepared statement con binding dei parametri: immunità dalle SQL injection.
- Validazione via espressione regolare di nomi di tabelle e colonne, non parametrizzabili.
- White-list delle colonne scrivibili in inserimento e aggiornamento.

8. Presentation layer (Smarty)

Il rendering è affidato a Smarty tramite il wrapper `SmartyView`, invocato dai controller con `view(template, dati)`. Il wrapper configura le directory, registra i modificatori di template e inietta in ogni pagina un insieme di variabili globali: stato della sessione, token CSRF, categorie di navigazione, contatore del carrello e variabili temporali. I template `.tpl` sono organizzati per area funzionale, con componenti riusabili (`components/`) e layout comuni (`layouts/`).

Difesa da Cross-Site Scripting (XSS)

Smarty è configurato con `setEscapeHtml(true)`: l'escaping HTML dell'output è applicato per default, neutralizzando l'iniezione di markup attraverso i dati forniti dagli utenti (p.es. il titolo di un annuncio).

9. Sicurezza

9.1 Gestione della sessione

`SessionManager` centralizza il ciclo di vita della sessione applicando le seguenti politiche:

- cookie **HttpOnly** e **SameSite=Lax**, con flag **Secure** automatico su HTTPS (attivo in produzione presso il provider di hosting);
- `session.use_strict_mode` attivo e accettazione dell'identificativo solo via cookie;
- rigenerazione dell'ID di sessione dopo l'autenticazione (mitigazione del *session fixation*);
- scadenza per inattività (30 minuti, configurabile) e invalidazione completa al logout.

9.2 Protezione CSRF

Ogni form che altera lo stato include un token sincronizzato, generato e associato alla sessione da `Csrf`. Il Front Controller lo valida su ogni POST mediante confronto a tempo costante (`hash_equals`); il fallimento produce una risposta 400.

9.3 Validazione degli upload

Il caricamento delle immagini degli annunci è vincolato a un massimo di cinque file da 3 MB ciascuno; il tipo MIME reale è verificato dal contenuto (`finfo`, ammessi JPG/PNG/WEBP), i file sono rinominati casualmente e confinati in `public/uploads/`, con controllo del percorso in cancellazione a difesa dal *path traversal*.

10. Modello dei dati

La base dati `nerdvault` è in larga parte in corrispondenza con le Entity. Le tabelle principali sono riassunte di seguito.

Tabella	Contenuto
<code>utente_registrato</code>	Utenti privati: credenziali, profilo, stato di verifica e ban.
<code>admin</code>	Amministratori, con livello di sicurezza (1 o 2).
<code>account_business</code>	Venditori professionali, associati a un utente.
<code>categoria</code>	Categorie merceologiche degli annunci.
<code>annuncio</code>	Annunci: titolo, prezzo, stato, categoria, proprietario (utente o business).
<code>immagine</code>	Immagini degli annunci (fino a cinque per annuncio).
<code>indirizzi</code>	Indirizzi di spedizione degli utenti e sedi dei business.
<code>preferito</code>	Wishlist: relazione utente-annuncio.
<code>carrello / elemento_carrello</code>	Carrello dell'utente e relativi articoli.
<code>pagamento</code>	Acquisti completati: importo, indirizzo, annuncio.
<code>feedback</code>	Recensioni post-acquisto.
<code>segnalazione</code>	Segnalazioni degli utenti con relativo stato di lavorazione.
<code>modera</code>	Registro di audit delle azioni di moderazione.
<code>password_reset</code>	Token temporanei per il recupero della password.

11. Deployment e ambiente di produzione

Il sistema, sviluppato in locale con XAMPP, è destinato alla pubblicazione presso un **provider di hosting** e all'accesso via Internet su dominio pubblico in HTTPS. L'architettura è concepita per rendere la transizione dallo sviluppo alla produzione una variazione di sola configurazione, senza modifiche al codice applicativo.

11.1 Configurazione separata dal codice

I parametri d'ambiente non sono versionati: il repository contiene solo i template `db.example.php` e `mail.example.php`, mentre i file effettivi `db.php` e `mail.php` sono creati per ciascun ambiente. In produzione i valori sensibili (host, credenziali del database, parametri SMTP) possono essere forniti tramite **variabili d'ambiente** (`DB_HOST`, `DB_NAME`, `DB_USERNAME`, `DB_PASSWORD`, ...), evitando di esporre segreti nel codice sorgente.

11.2 Esposizione e isolamento

- Il document root del web server punta esclusivamente a `public/`: le cartelle `src/`, `templates/` e la configurazione restano fuori dalla radice servita e non sono accessibili dal web.
- L'instradamento è gestito da `public/.htaccess`; in alternativa il provider può configurare un `rewrite` equivalente verso `index.php`.
- Le dipendenze sono installate con `composer install` e l'autoload è rigenerato in fase di rilascio.

11.3 Sicurezza in produzione

Sotto HTTPS il cookie di sessione assume automaticamente il flag `Secure`; dietro un proxy/terminazione TLS è possibile forzarlo con `SESSION_COOKIE_SECURE=1`. Il timeout di inattività della sessione è regolabile via `SESSION_IDLE_TIMEOUT`. La modalità di debug degli errori resta disattivata, così da non rivelare dettagli interni all'utente finale.

Sintesi del passaggio in produzione

Stesso codice, configurazione diversa: il deploy presso l'hosting si riduce a impostare le credenziali d'ambiente, far puntare il web server a `public/`, abilitare HTTPS e importare lo schema del database. Le difese già descritte (CSRF, escaping, prepared statement, cookie sicuri) diventano pienamente efficaci nel contesto pubblico.

Relazione tecnica sull'architettura di NerdVault. Il documento descrive l'organizzazione MVC a livelli, il meccanismo di Front Controller e routing, il livello di persistenza basato su Data Mapper, le misure di sicurezza e il deployment in ambiente di produzione presso un provider di hosting.